

E-mail Security

Module 2

Prepared by

Jesna J S

Assistant Professor CSE TKMCE

Topics Includes

- Pretty Good Privacy (PGP)
- Operational Description
- Cryptographic keys and key rings
- Message format
- PGP message generation
- PGP message reception
- Public key management

Pretty Good Privacy (PGP)

- **PGP (Pretty Good Privacy)** was developed primarily by Phil Zimmermann.
- It is considered a remarkable phenomenon because it was largely the effort of a **single individual** rather than a government or large organization.
- PGP provides **confidentiality and authentication services**.
- It is designed for:
 - ✓ Secure electronic mail
 - ✓ Secure file storage applications

Key Contributions of Phil Zimmermann

Zimmermann accomplished the following:

1. Careful Algorithm Selection

- Selected the **best available cryptographic algorithms**.
- Used them as secure building blocks for PGP.

2. Integrated System Design

- Combined different cryptographic techniques into a **general-purpose application**.
- Designed it to be:
 - ✓Independent of operating system
 - ✓Independent of processor type
- Made it user-friendly with a **small set of easy-to-use commands**.

3. Free Distribution

- Made the package available freely along with: documentation and source code
- Distributed via internet, bulletin boards and commercial networks such as AOL.

4. Commercial Agreement

- Entered into agreement with Viacrypt.
- Later associated with Network Associates.
- Enabled a fully compatible, low-cost commercial version of PGP.

Rapid Growth of PGP

PGP experienced explosive growth due to several reasons:

1. Worldwide Free Availability

- Available at no cost globally.
- Runs on multiple platforms including Windows, UNIX and Macintosh
- Commercial version provides vendor support for users who require it.

2. Strong and Publicly Reviewed Algorithms

- PGP includes widely trusted algorithms:
 - ✓ **Public-Key Algorithms: RSA, DSS and Diffie-Hellman**
 - ✓ **Symmetric-key Algorithms: CAST-128, IDEA and Triple DES**
 - ✓ **Hash Functions: SHA-1**
- These algorithms have undergone extensive public review and are considered highly secure.

3. Wide Range of Applicability

- Suitable for corporations enforcing standardized encryption.
- Useful for individuals communicating securely worldwide over the Internet.

4. Independence from Government Control

- Not developed by any governmental organization.
- Not controlled by any standards body.
- This independence increased trust among users skeptical of centralized authorities.

5. Internet Standards Recognition

- PGP is on the Internet standards track under:
 - RFC 3156 (MIME Security with OpenPGP).
- Despite standardization, PGP retains an “anti-establishment” image.

Table 18.1 Summary of PGP Services

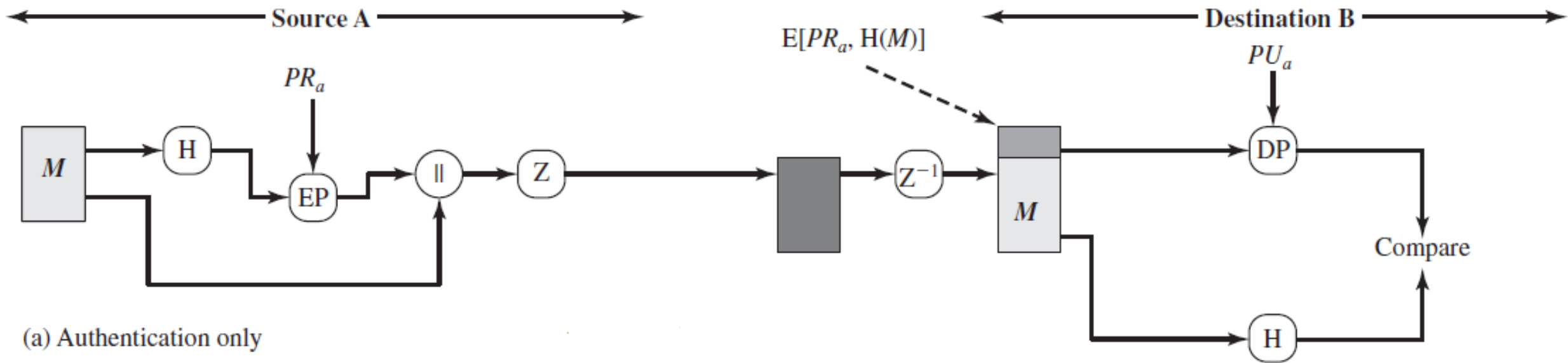
Function	Algorithms Used	Description
Digital signature	DSS/SHA or RSA/SHA	A hash code of a message is created using SHA-1. This message digest is encrypted using DSS or RSA with the sender's private key and included with the message.
Message encryption	CAST or IDEA or Three-key Triple DES with Diffie-Hellman or RSA	A message is encrypted using CAST-128 or IDEA or 3DES with a one-time session key generated by the sender. The session key is encrypted using Diffie-Hellman or RSA with the recipient's public key and included with the message.
Compression	ZIP	A message may be compressed for storage or transmission using ZIP.
E-mail compatibility	Radix-64 conversion	To provide transparency for e-mail applications, an encrypted message may be converted to an ASCII string using radix-64 conversion.

Operational Description

- K_s = session key used in symmetric encryption scheme
- PR_a = private key of user A, used in public-key encryption scheme
- PU_a = public key of user A, used in public-key encryption scheme
- EP = public-key encryption
- DP = public-key decryption
- EC = symmetric encryption
- DC = symmetric decryption

- H = hash function
- || = concatenation
- Z = compression using ZIP algorithm
- R64 = conversion to radix 64 ASCII format¹

1. AUTHENTICATION



1. The sender creates a message.
2. **SHA-1** is used to generate a **160-bit hash** code of the message.
3. The **hash code is encrypted** with **RSA** using the **sender's private key**, and the result is prepended to the message.
4. The receiver uses RSA with the **sender's public key** to decrypt and recover the hash code.
5. The receiver generates a **new hash code** for the message and compares it with the decrypted hash code. **If the two match, the message is accepted as authentic.**

- The combination of SHA-1 and RSA provides an effective digital signature scheme.

SHA-1:

- ✓ Generates a fixed-length hash (message digest) of the message.
- ✓ **Ensures message integrity.**
- ✓ Because of its strength, it is computationally infeasible to create another message that produces the same hash.

RSA:

- ✓ The sender encrypts the hash using their private key.
- ✓ Due to the strength of RSA only the possessor of the corresponding private key can generate that signature.
- ✓ The recipient verifies the signature using the sender's public key.
- ✓ **Ensures authentication and non-repudiation.**

The combination guarantees:

1. **Authentication (proof of sender identity)**
 2. **Integrity (message not altered)**
 3. **Non-repudiation (sender cannot deny signing)**
- As an alternative to RSA-based signatures, PGP supports Digital Signature Algorithm **(DSS) used together with SHA-1**
 - DSS/SHA-1 provides another standardized digital signature mechanism.

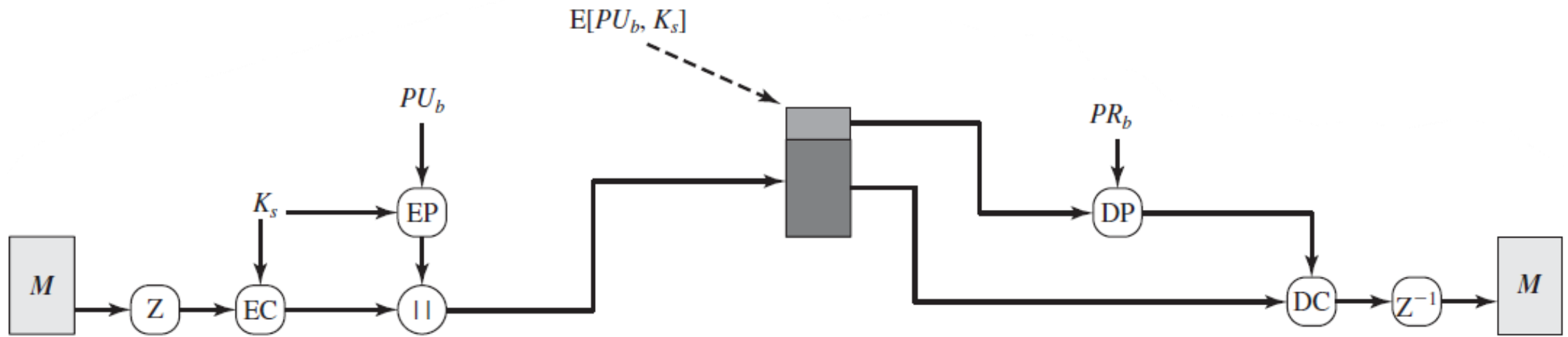
2. CONFIDENTIALITY

- Another basic service provided by PGP is **confidentiality**.
- It ensures:
 - ✓ Messages transmitted over networks remain secret.
 - ✓ Files stored locally remain protected from unauthorized access.
- Confidentiality is achieved through **encryption**.
- For message encryption, PGP uses symmetric cryptography. The following algorithms may be used: CAST-128, IDEA, Triple DES. Among these, CAST-128 is commonly used.
- PGP uses **64-bit Cipher Feedback (CFB) mode**

- **Key distribution is a major issue in symmetric encryption.**
- PGP solves this using a **session key mechanism.**

Characteristics of the Session Key

- ✓ A new **symmetric key** is generated for each message.
 - ✓ **The key is a random 128-bit number.**
 - ✓ **It is used only once.**
- Since the session key is needed for decryption:
 - ✓ It is bound to the message.
 - ✓ It is transmitted along with the encrypted message.
 - ✓ However, to protect it, the session key is encrypted using the receiver's public key.

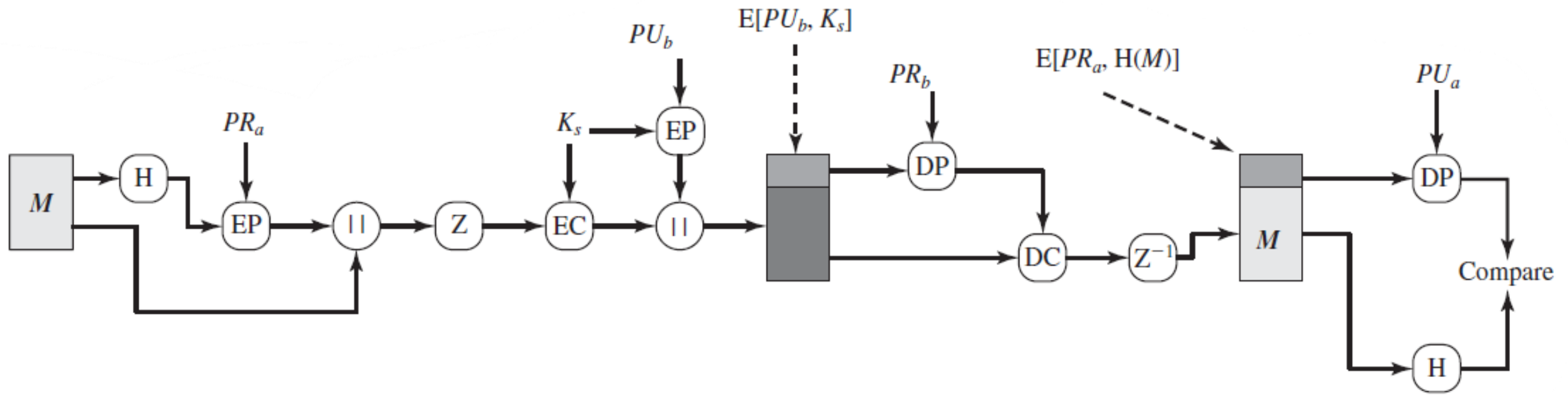


(b) Confidentiality only

1. The sender generates a message and a random 128-bit number to be used as session key for this message only.
2. The message is encrypted using CAST-128 (or IDEA or 3DES) with the session key.
3. The session key is encrypted with RSA using the recipient's public key and is prepended to the message.
4. The receiver uses RSA with its private key to decrypt and recover the session key.
5. The session key is used to decrypt the message.

3. Confidentiality and Authentication Together

- PGP allows both: **Confidentiality and Authentication** to be applied to the same message.



(c) Confidentiality and authentication

1. Generate a digital signature on the plaintext using the sender's private key.
2. Attach the signature to the plaintext message.
3. Generate a random session key.
4. Encrypt the message plus signature using a symmetric algorithm (e.g., CAST-128 / IDEA / Triple DES).
5. Encrypt the session key using the recipient's public key (e.g., RSA / ElGamal).
6. Send the encrypted session key and encrypted message to the recipient.

4. Compression

- PGP compresses the message after signing and before encryption. This saves: Email transmission space File storage space
- The correct order is:
- Sign the message
- Compress the message
- Encrypt the message
- Placement of compression is very important.

Why Signature is Generated Before Compression

(a) Storage Convenience

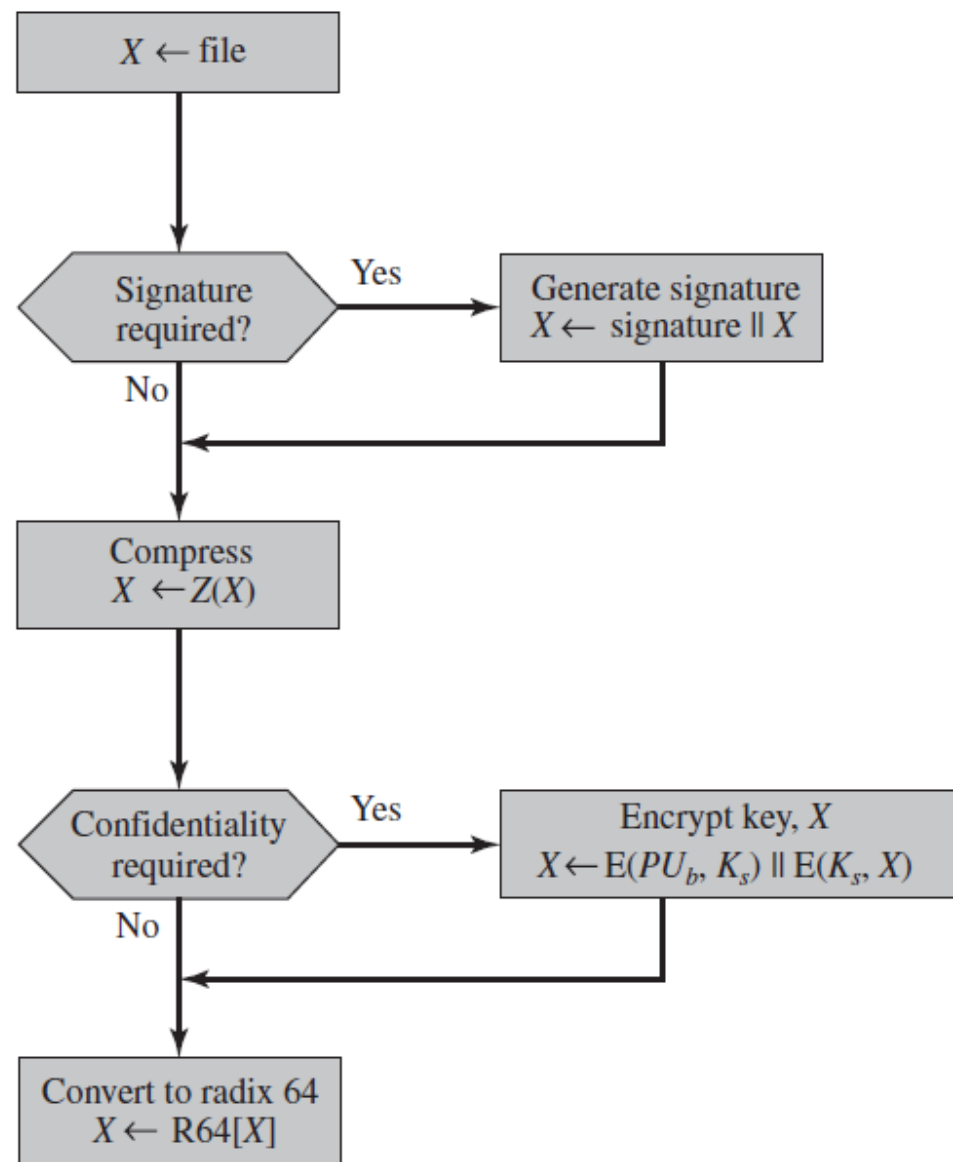
- It is better to sign the **uncompressed message**.
- Then only the uncompressed message and signature need to be stored.
- If signing were done after compression:
 - The compressed version must be stored, or
 - The message must be recompressed for verification.

(b) Compression is Not Deterministic

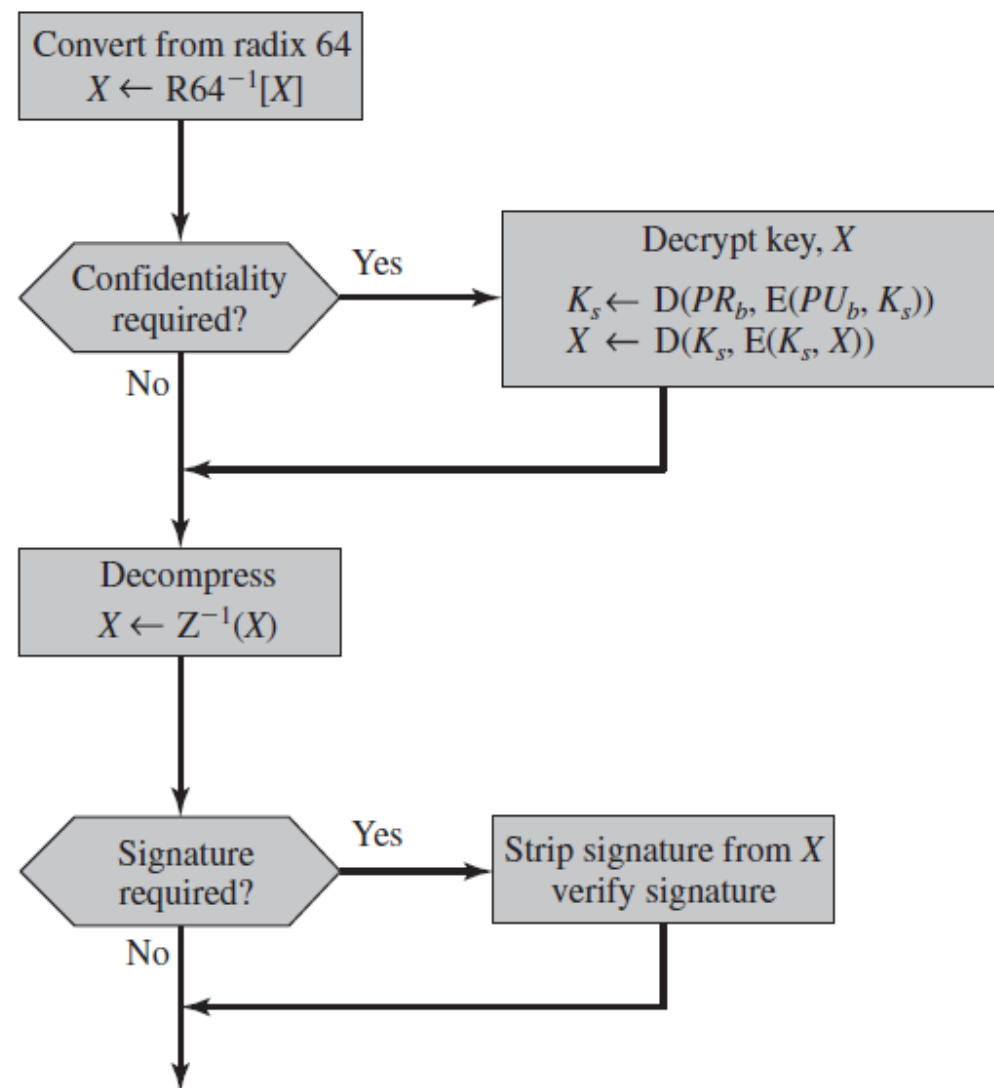
- PGP compression algorithm is **not deterministic**.
- Different implementations may produce different compressed outputs.
- They are interoperable (any version can decompress another).
- If signing were done after compression:
 - All implementations would need to use the same compression version.
- Therefore, signing is done before compression.

Why Encryption is Done After Compression

- Compression reduces redundancy in the message.
- Less redundancy makes cryptanalysis more difficult.
- Therefore, encryption after compression improves security.
- PGP uses ZIP compression algorithm.



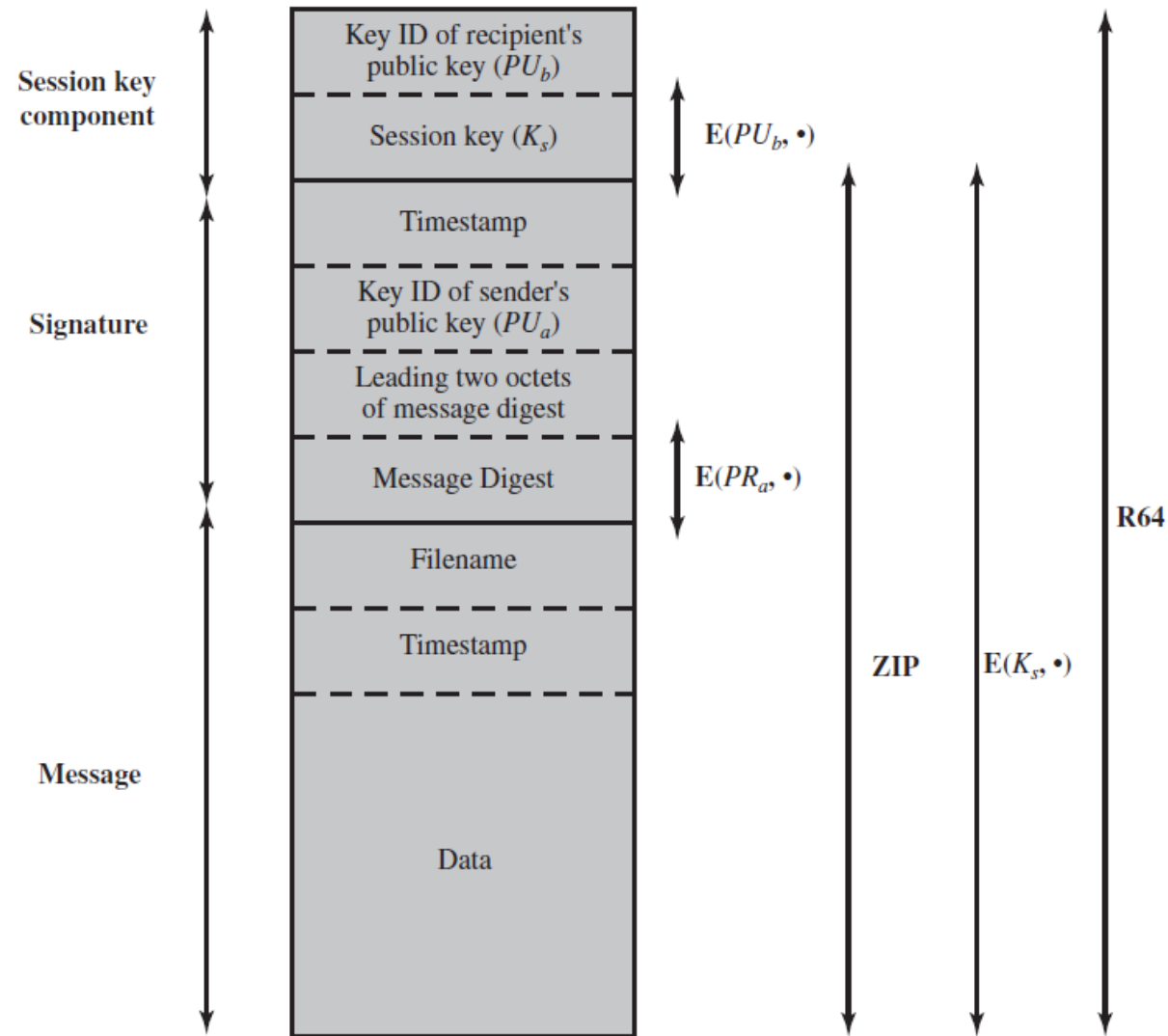
(a) Generic transmission diagram (from A)



(b) Generic reception diagram (to B)

Figure 18.2 Transmission and Reception of PGP Messages

Format of a PGP Message



Notation:

- $E(PU_b, \bullet)$ = encryption with user b's public key
- $E(PR_a, \bullet)$ = encryption with user a's private key
- $E(K_s, \bullet)$ = encryption with session key
- ZIP = Zip compression function
- R64 = Radix-64 conversion function

Figure 18.3 General Format PGP Message (from A to B)

Cryptographic Keys and Key Rings

- PGP uses **four types of cryptographic keys**:
 1. **One-time session symmetric keys**
 2. **Public keys**
 3. **Private keys**
 4. **Passphrase-based symmetric keys**
- Three main requirements exist regarding the use and management of these keys.

1. Requirement: Generation of Unpredictable Session Keys

- PGP requires a method to **generate unpredictable session keys**.
- **Session Key Characteristics**
 - ✓ A **session key** is associated with **one message only**.
 - ✓ It is used **once** to encrypt and decrypt that message.
 - ✓ Message encryption/decryption uses a **symmetric encryption algorithm**.

Session Key Generation

- A **128-bit random number** is generated to form the session key.
- The **CAST-128 encryption algorithm** itself is used to generate the random key.

Random Input Generation

- The plaintext input used in the generator comes from **randomized numbers** derived from user activity.
- Randomization uses:
 - ✓ **User keystrokes**
 - ✓ **Timing of keystrokes**
 - ✓ **Actual keys pressed**
- These inputs generate a **randomized data stream**.
- This stream is also combined with **previous session key output**
- These values together form the **key input for the generator**.
- Because CAST-128 strongly scrambles data, the generated **session keys become unpredictable**.

2. Requirement: Multiple Public/Private Key Pairs

- PGP allows a user to have **multiple public-key/private-key pairs**.
- **Reasons for Multiple Keys**
 1. **Key replacement over time**
 - A user may periodically change key pairs.
 - Messages already sent may still use old keys.
 - Recipients may only know the old public key until updates arrive.
 2. **Different communication groups**
 - Different key pairs can be used for different correspondents.
 3. **Security improvement**
 - Limits the amount of data encrypted with a single key.
- Therefore: **One user may have multiple public keys.**
- There is **not a one-to-one relationship between users and public keys.**
- Because of this, **a method to identify keys is required.**

- Each PGP user must maintain two files:
 - ✓ A file containing **their own public/private key pairs**
 - ✓ A file containing **public keys of correspondents**
- **These collections of keys are known as key rings.**

Key Identifiers (Key IDs)

- When a message is encrypted:
 - ✓ The **session key encrypts the message**
 - ✓ The **session key itself is encrypted with the recipient's public key**
- Only the recipient can decrypt the session key using their **private key**.
- **If a user has multiple key pairs, the recipient must know: Which key was used to encrypt the session key.**

How to solve it???

1. Send the Public Key with the Message

- The recipient could check if the key belongs to them.
- **Problem:** Public keys are **very large**
 - *Example: RSA public keys can be hundreds of digits*
 - Thus, this approach wastes space.

2. Use a Key Identifier

- Instead of sending the whole public key, **a short identifier (key ID) is sent.**
- However, this would require:
 1. **assigning identifiers**
 2. **storing mappings between key IDs and keys**
- This increases management overhead.

PGP Solution:

- **PGP automatically assigns a key ID to each public key.**

Key ID Generation

The key ID = least significant 64 bits of the public key.

- Therefore:
 - ✓ Each key ID is **64 bits long**
 - ✓ Probability of duplicate key IDs is **very small**
 - ✓ This key ID is used to identify the correct key.

Key ID in Digital Signatures

- A key ID is also included in digital signatures.
- Reason:
 - ✓ A sender may have **multiple private keys** for signing.
 - ✓ Thus, the recipient must know **which public key should verify the signature.**
- **Signature Component Includes**
 - **64-bit key ID**
 - **Used to identify the correct public key.**

When the message arrives:

- The recipient checks whether the **key ID matches a known public key** of the sender.
- Then the recipient **verifies the signature.**

Key Rings in PGP

- Key IDs are essential in the operation of PGP. A PGP message that provides **confidentiality and authentication** includes **two key IDs**.
- To manage these keys efficiently, **PGP stores them in organized data structures called key rings**.
- Each PGP user maintains **two key rings**:
 1. **Private-Key Ring**
 2. **Public-Key Ring**

Private-Key Ring

- The private-key ring stores all public/private key pairs owned by the user.
 - It can be viewed as a table, where each row represents one key pair.
1. **Timestamp**: Indicates the date and time when the key pair was generated.
 2. **Key ID** :The least significant 64 bits of the public key.
 3. **Public Key** :The public part of the key pair
 4. **Private Key**: The private part of the key pair(stored in encrypted form)
 5. **User ID**: Identifies the owner of the key pair, usually the user's email address

Private-Key Ring

Timestamp	Key ID*	Public Key	Encrypted Private Key	User ID*
• • •	• • •	• • •	• • •	• • •
T_i	$PU_i \bmod 2^{64}$	PU_i	$E(H(P_i), PR_i)$	User i
• • •	• • •	• • •	• • •	• • •

2. Public-Key Ring

- The **public-key ring** stores the **public keys of other users known to this user**.
- This allows the user to:
 - ✓ **encrypt messages for other users**
 - ✓ **verify digital signatures**

Public-Key Ring

Timestamp	Key ID*	Public Key	Owner Trust	User ID*	Key Legitimacy	Signature(s)	Signature Trust(s)
• • •	• • •	• • •	• • •	• • •	• • •	• • •	• • •
T_i	$PU_i \bmod 2^{64}$	PU_i	trust_flag_i	User i	trust_flag_i		
• • •	• • •	• • •	• • •	• • •	• • •	• • •	• • •

* = field used to index table

- **Fields in Public-Key Ring**

1. **Timestamp:** Date and time when the entry was created.
 2. **Key ID:** The **least significant 64 bits of the public key**.
 3. **Public Key:** The **public key of the corresponding user**.
 4. **User ID:** Identifies the owner of the public key. A single public key may have multiple User IDs.
- The public-key ring can also be indexed using:
 1. **User ID**
 2. **Key ID**
 - Both indexing methods are used for efficient key lookup.

Use of Key Rings

- Key rings are used during **message transmission and reception**.
- For simplicity, the description ignores:
 - *message compression*
 - *radix-64 conversion*
- During message transmission, if a message is **both signed and encrypted**, PGP uses the information stored in the **private-key ring and public-key ring** to perform the required operations.

PGP message generation

1. Signing the message:

- a. PGP retrieves the sender's private key from the private-key ring using `your_userid` as an index. If `your_userid` was not provided in the command, the first private key on the ring is retrieved.
- b. PGP prompts the user for the passphrase to recover the unencrypted private key.
- c. The signature component of the message is constructed.

2. Encrypting the message:

- a. PGP generates a session key and encrypts the message.
- b. PGP retrieves the recipient's public key from the public-key ring using `her_userid` as an index.
- c. The session key component of the message is constructed.

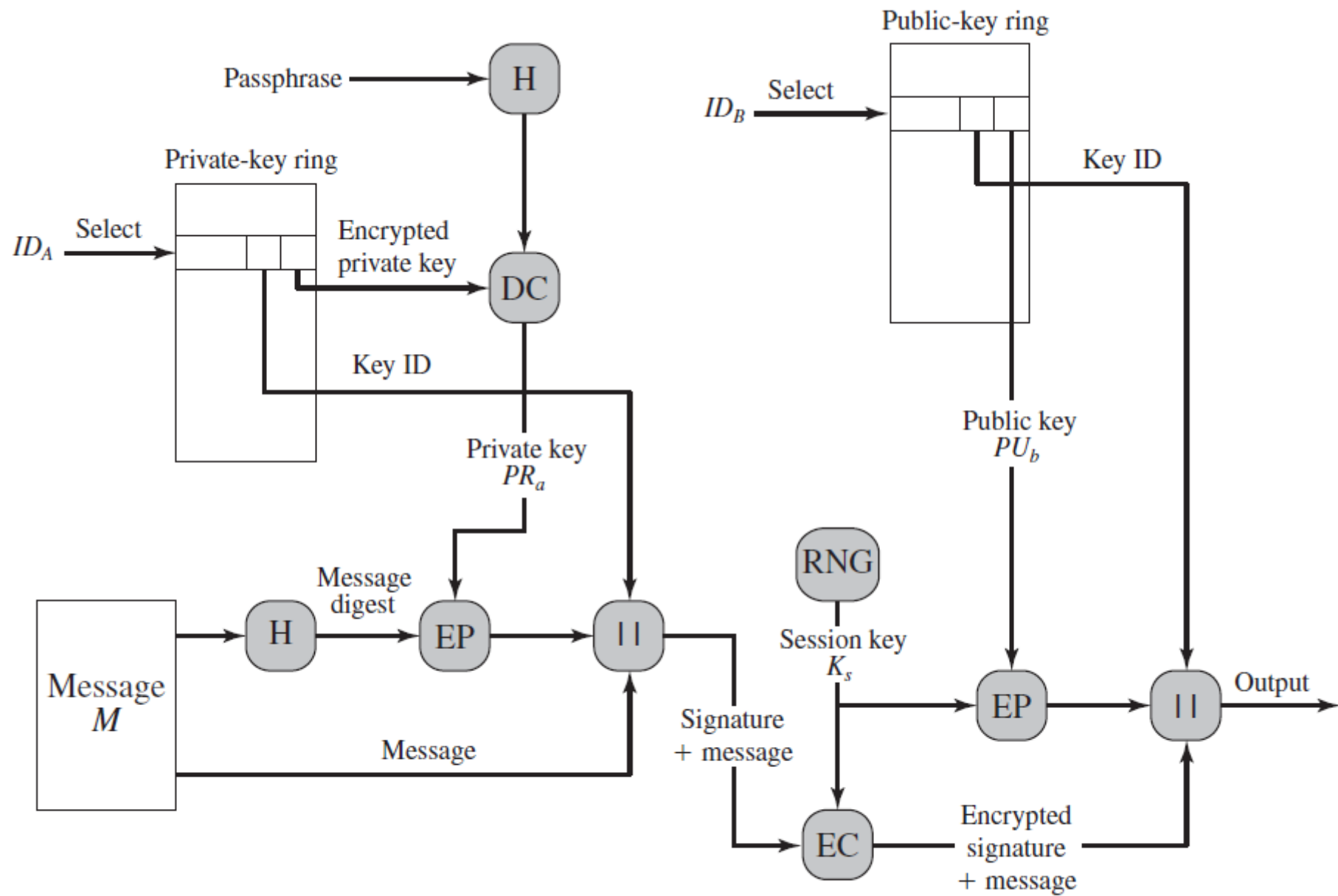


Figure 18.5 PGP Message Generation (from User A to User B: no compression or radix-64 conversion)

PGP message reception

1. Decrypting the message:

- a. PGP retrieves the receiver's private key from the private-key ring using the Key ID field in the session key component of the message as an index.
- b. PGP prompts the user for the passphrase to recover the unencrypted private key.
- c. PGP then recovers the session key and decrypts the message.

2. Authenticating the message:

- a. PGP retrieves the sender's public key from the public-key ring using the Key ID field in the signature key component of the message as an index.
- b. PGP recovers the transmitted message digest.
- c. PGP computes the message digest for the received message and compares it to the transmitted message digest to authenticate.

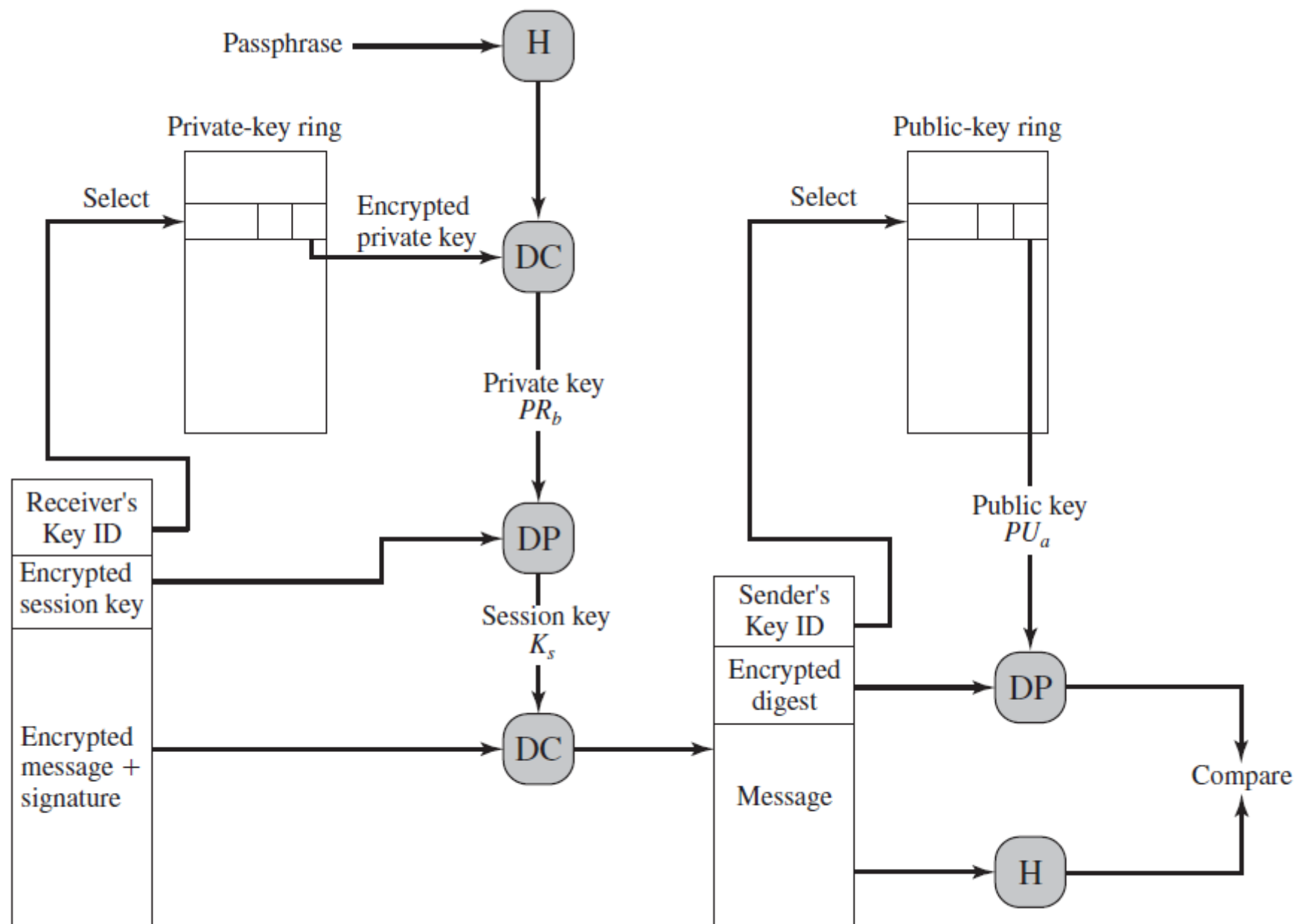


Figure 18.6 PGP Message Reception (from User A to User B; no compression or radix-64 conversion)

Public-Key Management in PGP

Need for Public-Key Management

- PGP provides **confidentiality and authentication using public-key cryptography**.
- However, the major challenge **is ensuring that public keys are genuine and not tampered with**.
- If a user stores a false public key in the public-key ring, two threats arise:
 - ✓ Attacker can forge digital signatures pretending to be another user.
 - ✓ Encrypted messages may be read by the attacker instead of the intended receiver.

Approaches to Obtain a Reliable Public Key

- To minimize the risk of false keys, PGP suggests several methods.

1. Physical Key Exchange

- ✓ The user directly obtains the public key from the owner (e.g., USB/floppy).
- ✓ Very secure but not practical for large networks.

2. Verification by Telephone

- ✓ The key owner sends the public key via email.
- ✓ A **fingerprint (SHA-1 digest)** of the key is generated.
- ✓ The fingerprint is verified over the phone.
- ✓ If fingerprints match → the key is authentic.

3. Trusted Introducer

- A **trusted person signs another user's public key.**
- This signed certificate contains:
 - ✓ Public key
 - ✓ Creation time
 - ✓ Validity period
- The introducer signs the certificate using their **private key**.
- Anyone who trusts the introducer can trust the signed key.

4. Certifying Authority

- A **trusted authority issues and signs certificates.**
- Users obtain the certificate from this authority.
- This approach is similar to **centralized PKI systems.**

Trust Concept in PGP

- PGP does not enforce a rigid certification authority system. Instead, it uses a **web-of-trust model** where users assign trust levels to others.
- Each public-key entry in the **public-key ring** contains trust information.

Trust Fields in PGP

- PGP maintains three main trust fields.

1. Owner Trust

- Assigned by the user. [Indicates **how much the user trusts this key owner to sign other keys.**]
- Possible values:
 - ✓Unknown
 - ✓Untrusted
 - ✓Marginally trusted
 - ✓Fully trusted
 - ✓Ultimately trusted

2. Signature Trust

- Indicates **how much the user trusts the signer of a certificate.**
- Derived from the **owner trust value** of the signer.

3. Key Legitimacy

- **Computed by PGP.**
- Indicates **how strongly a public key is believed to belong to a specific user.**
- Calculated using the **signature trust values** of the signatures attached to that key.

Trust Processing in PGP

When a new key is added to the public-key ring:

1. Assign Owner Trust

- If the key belongs to the user → ultimate trust.
- Otherwise, the user assigns a trust level.

2. Assign Signature Trust

- When a signature is added, PGP checks if the signer is known.
- If known → signer's owner trust becomes the signature trust.

3. Compute Key Legitimacy

- If at least one **fully trusted signature** exists → key is legitimate.
- Otherwise, PGP combines multiple trusted signatures.

Thus, multiple partially trusted signatures can collectively make a key legitimate.

Maintaining Consistency

PGP periodically processes the public-key ring:

- Updates **signature trust values** from owner trust values.
- Recomputes **key legitimacy** based on attached signatures.
- This ensures consistency in the trust relationships.

Revoking Public Keys

- A user may revoke a public key if:
 - ✓ The private key is compromised
 - ✓ The key is old or no longer secure

Key Revocation Certificate

- The owner creates a **revocation certificate**.
- It is **signed using the corresponding private key**.
- The certificate is distributed so others can update their key rings.
- Even if an attacker issues a revocation certificate, it only **disables the key**, which reduces the risk compared to misuse of the stolen key.